

not have to readjust to compensate for the change in the distribution of  $x$ .

Fixed distribution of inputs to a sub-network would have positive consequences for the layers *outside* the sub-network, as well. Consider a layer with a sigmoid activation function  $z = g(Wu + b)$  where  $u$  is the layer input, the weight matrix  $W$  and bias vector  $b$  are the layer parameters to be learned, and  $g(x) = \frac{1}{1+\exp(-x)}$ . As  $|x|$  increases,  $g'(x)$  tends to zero. This means that for all dimensions of  $x = Wu + b$  except those with small absolute values, the gradient flowing down to  $u$  will vanish and the model will train slowly. However, since  $x$  is affected by  $W$ ,  $b$  and the parameters of all the layers below, changes to those parameters during training will likely move many dimensions of  $x$  into the saturated regime of the nonlinearity and slow down the convergence. This effect is amplified as the network depth increases. In practice, the saturation problem and the resulting vanishing gradients are usually addressed by using Rectified Linear Units (Nair & Hinton, 2010)  $ReLU(x) = \max(x, 0)$ , careful initialization (Bengio & Glorot, 2010; Saxe et al., 2013), and small learning rates. If, however, we could ensure that the distribution of nonlinearity inputs remains more stable as the network trains, then the optimizer would be less likely to get stuck in the saturated regime, and the training would accelerate.

We refer to the change in the distributions of internal nodes of a deep network, in the course of training, as *Internal Covariate Shift*. Eliminating it offers a promise of faster training. We propose a new mechanism, which we call *Batch Normalization*, that takes a step towards reducing internal covariate shift, and in doing so dramatically accelerates the training of deep neural nets. It accomplishes this via a normalization step that fixes the means and variances of layer inputs. Batch Normalization also has a beneficial effect on the gradient flow through the network, by reducing the dependence of gradients on the scale of the parameters or of their initial values. This allows us to use much higher learning rates without the risk of divergence. Furthermore, batch normalization regularizes the model and reduces the need for Dropout (Srivastava et al., 2014). Finally, Batch Normalization makes it possible to use saturating nonlinearities by preventing the network from getting stuck in the saturated modes.

In Sec. 4.2, we apply Batch Normalization to the best-performing ImageNet classification network, and show that we can match its performance using only 7% of the training steps, and can further exceed its accuracy by a substantial margin. Using an ensemble of such networks trained with Batch Normalization, we achieve the top-5 error rate that improves upon the best known results on ImageNet classification.

## 2 Towards Reducing Internal Covariate Shift

We define *Internal Covariate Shift* as the change in the distribution of network activations due to the change in network parameters during training. To improve the training, we seek to reduce the internal covariate shift. By fixing the distribution of the layer inputs  $x$  as the training progresses, we expect to improve the training speed. It has been long known (LeCun et al., 1998b; Wiesler & Ney, 2011) that the network training converges faster if its inputs are whitened – i.e., linearly transformed to have zero means and unit variances, and decorrelated. As each layer observes the inputs produced by the layers below, it would be advantageous to achieve the same whitening of the inputs of each layer. By whitening the inputs to each layer, we would take a step towards achieving the fixed distributions of inputs that would remove the ill effects of the internal covariate shift.

We could consider whitening activations at every training step or at some interval, either by modifying the network directly or by changing the parameters of the optimization algorithm to depend on the network activation values (Wiesler et al., 2014; Raiko et al., 2012; Povey et al., 2014; Desjardins & Kavukcuoglu). However, if these modifications are interspersed with the optimization steps, then the gradient descent step may attempt to update the parameters in a way that requires the normalization to be updated, which reduces the effect of the gradient step. For example, consider a layer with the input  $u$  that adds the learned bias  $b$ , and normalizes the result by subtracting the mean of the activation computed over the training data:  $\hat{x} = x - E[x]$  where  $x = u + b$ ,  $\mathcal{X} = \{x_{1..N}\}$  is the set of values of  $x$  over the training set, and  $E[x] = \frac{1}{N} \sum_{i=1}^N x_i$ . If a gradient descent step ignores the dependence of  $E[x]$  on  $b$ , then it will update  $b \leftarrow b + \Delta b$ , where  $\Delta b \propto -\partial \ell / \partial \hat{x}$ . Then  $u + (b + \Delta b) - E[u + (b + \Delta b)] = u + b - E[u + b]$ . Thus, the combination of the update to  $b$  and subsequent change in normalization led to no change in the output of the layer nor, consequently, the loss. As the training continues,  $b$  will grow indefinitely while the loss remains fixed. This problem can get worse if the normalization not only centers but also scales the activations. We have observed this empirically in initial experiments, where the model blows up when the normalization parameters are computed outside the gradient descent step.

The issue with the above approach is that the gradient descent optimization does not take into account the fact that the normalization takes place. To address this issue, we would like to ensure that, for any parameter values, the network *always* produces activations with the desired distribution. Doing so would allow the gradient of the loss with respect to the model parameters to account for the normalization, and for its dependence on the model parameters  $\Theta$ . Let again  $x$  be a layer input, treated as a

vector, and  $\mathcal{X}$  be the set of these inputs over the training data set. The normalization can then be written as a transformation

$$\hat{x} = \text{Norm}(x, \mathcal{X})$$

which depends not only on the given training example  $x$  but on all examples  $\mathcal{X}$  – each of which depends on  $\Theta$  if  $x$  is generated by another layer. For backpropagation, we would need to compute the Jacobians

$$\frac{\partial \text{Norm}(x, \mathcal{X})}{\partial x} \text{ and } \frac{\partial \text{Norm}(x, \mathcal{X})}{\partial \mathcal{X}};$$

ignoring the latter term would lead to the explosion described above. Within this framework, whitening the layer inputs is expensive, as it requires computing the covariance matrix  $\text{Cov}[x] = \mathbb{E}_{x \in \mathcal{X}}[xx^T] - \mathbb{E}[x]\mathbb{E}[x]^T$  and its inverse square root, to produce the whitened activations  $\text{Cov}[x]^{-1/2}(x - \mathbb{E}[x])$ , as well as the derivatives of these transforms for backpropagation. This motivates us to seek an alternative that performs input normalization in a way that is differentiable and does not require the analysis of the entire training set after every parameter update.

Some of the previous approaches (e.g. (Lyu & Simoncelli, 2008)) use statistics computed over a single training example, or, in the case of image networks, over different feature maps at a given location. However, this changes the representation ability of a network by discarding the absolute scale of activations. We want to preserve the information in the network, by normalizing the activations in a training example relative to the statistics of the entire training data.

### 3 Normalization via Mini-Batch Statistics

Since the full whitening of each layer’s inputs is costly and not everywhere differentiable, we make two necessary simplifications. The first is that instead of whitening the features in layer inputs and outputs jointly, we will normalize each scalar feature independently, by making it have the mean of zero and the variance of 1. For a layer with  $d$ -dimensional input  $x = (x^{(1)} \dots x^{(d)})$ , we will normalize each dimension

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

where the expectation and variance are computed over the training data set. As shown in (LeCun et al., 1998b), such normalization speeds up convergence, even when the features are not decorrelated.

Note that simply normalizing each input of a layer may change what the layer can represent. For instance, normalizing the inputs of a sigmoid would constrain them to the linear regime of the nonlinearity. To address this, we make sure that *the transformation inserted in the network can represent the identity transform*. To accomplish this,

we introduce, for each activation  $x^{(k)}$ , a pair of parameters  $\gamma^{(k)}, \beta^{(k)}$ , which scale and shift the normalized value:

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}.$$

These parameters are learned along with the original model parameters, and restore the representation power of the network. Indeed, by setting  $\gamma^{(k)} = \sqrt{\text{Var}[x^{(k)}]}$  and  $\beta^{(k)} = \mathbb{E}[x^{(k)}]$ , we could recover the original activations, if that were the optimal thing to do.

In the batch setting where each training step is based on the entire training set, we would use the whole set to normalize activations. However, this is impractical when using stochastic optimization. Therefore, we make the second simplification: since we use mini-batches in stochastic gradient training, *each mini-batch produces estimates of the mean and variance* of each activation. This way, the statistics used for normalization can fully participate in the gradient backpropagation. Note that the use of mini-batches is enabled by computation of per-dimension variances rather than joint covariances; in the joint case, regularization would be required since the mini-batch size is likely to be smaller than the number of activations being whitened, resulting in singular covariance matrices.

Consider a mini-batch  $\mathcal{B}$  of size  $m$ . Since the normalization is applied to each activation independently, let us focus on a particular activation  $x^{(k)}$  and omit  $k$  for clarity. We have  $m$  values of this activation in the mini-batch,

$$\mathcal{B} = \{x_{1\dots m}\}.$$

Let the normalized values be  $\hat{x}_{1\dots m}$ , and their linear transformations be  $y_{1\dots m}$ . We refer to the transform

$$\text{BN}_{\gamma, \beta} : x_{1\dots m} \rightarrow y_{1\dots m}$$

as the *Batch Normalizing Transform*. We present the BN Transform in Algorithm 1. In the algorithm,  $\epsilon$  is a constant added to the mini-batch variance for numerical stability.

|                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Input:</b> Values of $x$ over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$ ;<br>Parameters to be learned: $\gamma, \beta$ |
| <b>Output:</b> $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$                                                                      |
| $\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$                                   |
| $\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$  |
| $\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$     |
| $y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$               |

**Algorithm 1:** Batch Normalizing Transform, applied to activation  $x$  over a mini-batch.

The BN transform can be added to a network to manipulate any activation. In the notation  $y = \text{BN}_{\gamma, \beta}(x)$ , we

indicate that the parameters  $\gamma$  and  $\beta$  are to be learned, but it should be noted that the BN transform does not independently process the activation in each training example. Rather,  $\text{BN}_{\gamma,\beta}(x)$  depends both on the training example *and the other examples in the mini-batch*. The scaled and shifted values  $y$  are passed to other network layers. The normalized activations  $\hat{x}$  are internal to our transformation, but their presence is crucial. The distributions of values of any  $\hat{x}$  has the expected value of 0 and the variance of 1, as long as the elements of each mini-batch are sampled from the same distribution, and if we neglect  $\epsilon$ . This can be seen by observing that  $\sum_{i=1}^m \hat{x}_i = 0$  and  $\frac{1}{m} \sum_{i=1}^m \hat{x}_i^2 = 1$ , and taking expectations. Each normalized activation  $\hat{x}^{(k)}$  can be viewed as an input to a sub-network composed of the linear transform  $y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$ , followed by the other processing done by the original network. These sub-network inputs all have fixed means and variances, and although the joint distribution of these normalized  $\hat{x}^{(k)}$  can change over the course of training, we expect that the introduction of normalized inputs accelerates the training of the sub-network and, consequently, the network as a whole.

During training we need to backpropagate the gradient of loss  $\ell$  through this transformation, as well as compute the gradients with respect to the parameters of the BN transform. We use chain rule, as follows (before simplification):

$$\begin{aligned}\frac{\partial \ell}{\partial \hat{x}_i} &= \frac{\partial \ell}{\partial y_i} \cdot \gamma \\ \frac{\partial \ell}{\partial \sigma_B^2} &= \sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot (x_i - \mu_B) \cdot \frac{-1}{2} (\sigma_B^2 + \epsilon)^{-3/2} \\ \frac{\partial \ell}{\partial \mu_B} &= \left( \sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_B^2 + \epsilon}} \right) + \frac{\partial \ell}{\partial \sigma_B^2} \cdot \frac{\sum_{i=1}^m -2(x_i - \mu_B)}{m} \\ \frac{\partial \ell}{\partial x_i} &= \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_B^2} \cdot \frac{2(x_i - \mu_B)}{m} + \frac{\partial \ell}{\partial \mu_B} \cdot \frac{1}{m} \\ \frac{\partial \ell}{\partial \gamma} &= \sum_{i=1}^m \frac{\partial \ell}{\partial y_i} \cdot \hat{x}_i \\ \frac{\partial \ell}{\partial \beta} &= \sum_{i=1}^m \frac{\partial \ell}{\partial y_i}\end{aligned}$$

Thus, BN transform is a differentiable transformation that introduces normalized activations into the network. This ensures that as the model is training, layers can continue learning on input distributions that exhibit less internal covariate shift, thus accelerating the training. Furthermore, the learned affine transform applied to these normalized activations allows the BN transform to represent the identity transformation and preserves the network capacity.

### 3.1 Training and Inference with Batch-Normalized Networks

To *Batch-Normalize* a network, we specify a subset of activations and insert the BN transform for each of them, according to Alg. 1. Any layer that previously received  $x$  as the input, now receives  $\text{BN}(x)$ . A model employing Batch Normalization can be trained using batch gradient descent, or Stochastic Gradient Descent with a mini-batch size  $m > 1$ , or with any of its variants such as Adagrad

(Duchi et al., 2011). The normalization of activations that depends on the mini-batch allows efficient training, but is neither necessary nor desirable during inference; we want the output to depend only on the input, deterministically. For this, once the network has been trained, we use the normalization

$$\hat{x} = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}}$$

using the population, rather than mini-batch, statistics. Neglecting  $\epsilon$ , these normalized activations have the same mean 0 and variance 1 as during training. We use the unbiased variance estimate  $\text{Var}[x] = \frac{m}{m-1} \cdot \mathbb{E}_B[\sigma_B^2]$ , where the expectation is over training mini-batches of size  $m$  and  $\sigma_B^2$  are their sample variances. Using moving averages instead, we can track the accuracy of a model as it trains. Since the means and variances are fixed during inference, the normalization is simply a linear transform applied to each activation. It may further be composed with the scaling by  $\gamma$  and shift by  $\beta$ , to yield a single linear transform that replaces  $\text{BN}(x)$ . Algorithm 2 summarizes the procedure for training batch-normalized networks.

**Input:** Network  $N$  with trainable parameters  $\Theta$ ;  
subset of activations  $\{x^{(k)}\}_{k=1}^K$   
**Output:** Batch-normalized network for inference,  $N_{\text{BN}}^{\text{inf}}$

- 1:  $N_{\text{BN}}^{\text{tr}} \leftarrow N$  // Training BN network
- 2: **for**  $k = 1 \dots K$  **do**
- 3:   Add transformation  $y^{(k)} = \text{BN}_{\gamma^{(k)}, \beta^{(k)}}(x^{(k)})$  to  $N_{\text{BN}}^{\text{tr}}$  (Alg. 1)
- 4:   Modify each layer in  $N_{\text{BN}}^{\text{tr}}$  with input  $x^{(k)}$  to take  $y^{(k)}$  instead
- 5: **end for**
- 6: Train  $N_{\text{BN}}^{\text{tr}}$  to optimize the parameters  $\Theta \cup \{\gamma^{(k)}, \beta^{(k)}\}_{k=1}^K$
- 7:  $N_{\text{BN}}^{\text{inf}} \leftarrow N_{\text{BN}}^{\text{tr}}$  // Inference BN network with frozen parameters
- 8: **for**  $k = 1 \dots K$  **do**
- 9:   // For clarity,  $x \equiv x^{(k)}, \gamma \equiv \gamma^{(k)}, \mu_B \equiv \mu_B^{(k)}$ , etc.
- 10:   Process multiple training mini-batches  $\mathcal{B}$ , each of size  $m$ , and average over them:
 
$$\mathbb{E}[x] \leftarrow \mathbb{E}_B[\mu_B]$$

$$\text{Var}[x] \leftarrow \frac{m}{m-1} \mathbb{E}_B[\sigma_B^2]$$
- 11:   In  $N_{\text{BN}}^{\text{inf}}$ , replace the transform  $y = \text{BN}_{\gamma, \beta}(x)$  with
 
$$y = \frac{\gamma}{\sqrt{\text{Var}[x] + \epsilon}} \cdot x + \left( \beta - \frac{\gamma \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} \right)$$
- 12: **end for**

**Algorithm 2:** Training a Batch-Normalized Network

### 3.2 Batch-Normalized Convolutional Networks

Batch Normalization can be applied to any set of activations in the network. Here, we focus on transforms

| Model                    | Resolution | Crops | Models | Top-1 error | Top-5 error  |
|--------------------------|------------|-------|--------|-------------|--------------|
| GoogLeNet ensemble       | 224        | 144   | 7      | -           | 6.67%        |
| Deep Image low-res       | 256        | -     | 1      | -           | 7.96%        |
| Deep Image high-res      | 512        | -     | 1      | 24.88       | 7.42%        |
| Deep Image ensemble      | variable   | -     | -      | -           | 5.98%        |
| BN-Inception single crop | 224        | 1     | 1      | 25.2%       | 7.82%        |
| BN-Inception multicrop   | 224        | 144   | 1      | 21.99%      | 5.82%        |
| BN-Inception ensemble    | 224        | 144   | 6      | 20.1%       | <b>4.9%*</b> |

Figure 4: *Batch-Normalized Inception comparison with previous state of the art on the provided validation set comprising 50000 images. \*BN-Inception ensemble has reached 4.82% top-5 error on the 100000 images of the test set of the ImageNet as reported by the test server.*

plies to sub-networks and layers, and removing it from internal activations of the network may aid in training. Our proposed method draws its power from normalizing activations, and from incorporating this normalization in the network architecture itself. This ensures that the normalization is appropriately handled by any optimization method that is being used to train the network. To enable stochastic optimization methods commonly used in deep network training, we perform the normalization for each mini-batch, and backpropagate the gradients through the normalization parameters. Batch Normalization adds only two extra parameters per activation, and in doing so preserves the representation ability of the network. We presented an algorithm for constructing, training, and performing inference with batch-normalized networks. The resulting networks can be trained with saturating nonlinearities, are more tolerant to increased training rates, and often do not require Dropout for regularization.

Merely adding Batch Normalization to a state-of-the-art image classification model yields a substantial speedup in training. By further increasing the learning rates, removing Dropout, and applying other modifications afforded by Batch Normalization, we reach the previous state of the art with only a small fraction of training steps – and then beat the state of the art in single-network image classification. Furthermore, by combining multiple models trained with Batch Normalization, we perform better than the best known system on ImageNet, by a significant margin.

Interestingly, our method bears similarity to the standardization layer of (Gülçehre & Bengio, 2013), though the two methods stem from very different goals, and perform different tasks. The goal of Batch Normalization is to achieve a stable distribution of activation values throughout training, and in our experiments we apply it before the nonlinearity since that is where matching the first and second moments is more likely to result in a stable distribution. On the contrary, (Gülçehre & Bengio, 2013) apply the standardization layer to the *output* of the nonlinearity, which results in sparser activations. In our large-scale image classification experiments, we have not observed the nonlinearity *inputs* to be sparse, neither with nor without Batch Normalization. Other notable differ-

entiating characteristics of Batch Normalization include the learned scale and shift that allow the BN transform to represent identity (the standardization layer did not require this since it was followed by the learned linear transform that, conceptually, absorbs the necessary scale and shift), handling of convolutional layers, deterministic inference that does not depend on the mini-batch, and batch-normalizing each convolutional layer in the network.

In this work, we have not explored the full range of possibilities that Batch Normalization potentially enables. Our future work includes applications of our method to Recurrent Neural Networks (Pascanu et al., 2013), where the internal covariate shift and the vanishing or exploding gradients may be especially severe, and which would allow us to more thoroughly test the hypothesis that normalization improves gradient propagation (Sec. 3.3). We plan to investigate whether Batch Normalization can help with domain adaptation, in its traditional sense – i.e. whether the normalization performed by the network would allow it to more easily generalize to new data distributions, perhaps with just a recomputation of the population means and variances (Alg. 2). Finally, we believe that further theoretical analysis of the algorithm would allow still more improvements and applications.

## References

- Bengio, Yoshua and Glorot, Xavier. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of AISTATS 2010*, volume 9, pp. 249–256, May 2010.
- Dean, Jeffrey, Corrado, Greg S., Monga, Rajat, Chen, Kai, Devin, Matthieu, Le, Quoc V., Mao, Mark Z., Ranzato, Marc’Aurelio, Senior, Andrew, Tucker, Paul, Yang, Ke, and Ng, Andrew Y. Large scale distributed deep networks. In *NIPS*, 2012.
- Desjardins, Guillaume and Kavukcuoglu, Koray. Natural neural networks. (unpublished).
- Duchi, John, Hazan, Elad, and Singer, Yoram. Adaptive subgradient methods for online learning and stochastic